

VIRTUSA CODING — PYTHON → JAVA STUDY GUIDE

This guide converts the code shown in the uploaded VIRTUSA coding-answer PDF into Java. The uploaded source is largely screenshot-based and contains function-style solutions rather than complete console programs. Therefore, the Java below is written as reusable methods. Input/output examples are illustrative unless the source explicitly supplied them.

Important: a few screenshots are low-resolution or partially unclear. Where the visible source logic was not fully readable, the guide preserves the visible structure and labels the example as illustrative rather than inventing a problem statement.

Quick Java reminders

```
Array sorting: Arrays.sort(arr);
String length: s.length()
Array length: arr.length
2D access: arr[i][j]
Hash set: HashSet<Integer> set = new HashSet<>();
Hash map: HashMap<Character,Integer> map = new HashMap<>();
String → char[]: s.toCharArray()
Character ASCII: (int)ch
Absolute value: Math.abs(x)
Minimum/maximum: Math.min(a,b), Math.max(a,b)
```

1) Pizza Party

Description: Find the first divisor of input1 in the range [input2, input1] and return the sum of its decimal digits.

Input: input1 = 12, input2 = 5 **Output:** Output: 3 (6 is the first divisor; 6 → 6).

One-line explanation: Loop through possible divisors; once one is found, add its digits.

```
static int pizzaParty(int input1, int input2) {
    for (int i = input2; i <= input1; i++) {
        if (input1 % i == 0) {
            int n = i, sum = 0;
            while (n > 0) {
                sum += n % 10;
                n /= 10;
            }
            return sum;
        }
    }
    return 0;
}
```

2) Bounty

Description: Count input1-digit numbers containing at least two zeroes, returning the count modulo 1,000,000,007.

Input: input1 = 2 **Output:** Output: 0.

One-line explanation: Convert each number to a string and count its zeroes.

```
static long bounty(int input1) {
    long cnt = 0;
    long mod = 1_000_000_007L;
    int start = (int)Math.pow(10, input1 - 1);
    int end = (int)Math.pow(10, input1);

    for (int i = start; i < end; i++) {
        String s = String.valueOf(i);
        int zeros = 0;
        for (char ch : s.toCharArray())
            if (ch == '0') zeros++;
        if (zeros >= 2) cnt++;
    }
    return cnt % mod;
}
```

3) Find Commas

Description: Count the total number of commas written when numbers from 1 through N are formatted with thousands separators.

Input: input1 = 1000 **Output:** Output: 1.

One-line explanation: For 1,000–999,999 each number has one comma; for 1,000,000 onward the count increases.

```
static long findCommas(long input1) {
    long ans = input1, cur = 1000, result = 0, cnt = 1;

    while (cur <= ans) {
        long next = cur * 1000;
        long num = Math.min(ans - cur + 1, next - cur);
        result += num * cnt;
        cur = next;
        cnt++;
    }
    return result;
}
```

4) Find X

Description: Build a number from bit positions that occur in at least two of three transformed inputs.

Input: input1=7, input2=7, input3=7 **Output:** Output: 1 (illustrative).

One-line explanation: Check each bit in all three values and keep it when at least two have that bit set.

```
static int findX(int input1, int input2, int input3) {
    int x1 = (input1 - 5) / 2;
    int x2 = (input2 - 5) / 2;
    int x3 = (input3 - 5) / 2;
    int ans = 0;

    for (int i = 0; i < 31; i++) {
        int count = 0;
        if ((x1 & (1 << i)) != 0) count++;
        if ((x2 & (1 << i)) != 0) count++;
        if ((x3 & (1 << i)) != 0) count++;
        if (count >= 2) ans |= (1 << i);
    }
    return ans;
}
```

5) Nearest Corner

Description: Given a target character in a string, find the smaller distance to the nearest '-' character on either side.

Input: target='X', s="abXc-def" **Output:** Output: 1.

One-line explanation: Walk left and right from the target until '-' is reached, then take the smaller count.

```
static int nearestCorner(char target, String s) {
    int idx = s.indexOf(target);
    int left = 0, right = 0;

    for (int i = idx - 1; i >= 0; i--) {
        if (s.charAt(i) == '-') break;
        left++;
    }
    for (int i = idx + 1; i < s.length(); i++) {
        if (s.charAt(i) == '-') break;
        right++;
    }
    return Math.min(left, right);
}
```

6) Frog in the Pond

Description: For every position, add the flower value and symmetric values at equal jump distances; return the maximum total.

Input: arr = [1,2,5,2,1] **Output:** Output: 10 (illustrative).

One-line explanation: Use the center position and add matching left/right positions.

```
static int frogInThePond(int[] a) {
    int n = a.length, max = 0;

    for (int i = 0; i < n; i++) {
        int petal = a[i];
        int jumps = Math.min(Math.min(i, n - i - 1), n - 2);

        for (int j = 1; j <= jumps; j++)
            petal += a[i + j] + a[i - j];

        max = Math.max(max, petal);
    }
    return max;
}
```

7) Chess Game

Description: Accumulate $\text{ceil}(\text{input2}/2)$ on even rounds and $\text{floor}(\text{input2}/2)$ on odd rounds for input1 rounds.

Input: $\text{input1}=3, \text{input2}=5$ **Output:** Output: 7.

One-line explanation: Integer arithmetic gives $\text{ceil}(n/2)$ as $(n+1)/2$ and $\text{floor}(n/2)$ as $n/2$.

```
static int chessGame(int input1, int input2) {
    int cnt = 0;

    for (int i = 0; i < input1; i++) {
        if (i % 2 == 0)
            cnt += (input2 + 1) / 2;
        else
            cnt += input2 / 2;
    }
    return cnt;
}
```

8) Ant on Rail

Description: Track a running position; every time it returns to zero, increment the count.

Input: $\text{moves} = [1,-1,2,-2]$ **Output:** Output: 2.

One-line explanation: Keep a running position and count how many times it reaches zero.

```
static int rearrangeSeats(int[] moves) {
    int pos = 0, sta = 0;

    for (int x : moves) {
        pos += x;
        if (pos == 0)
            sta++;
    }
    return sta;
}
```

9) Max Subarray Code

Description: For every fixed-size window, calculate the weighted score $\text{sum}((\text{index}+1)*\text{value})$ and return the maximum.

Input: arr=[1,2,3], window=2 **Output:** Output: 8 (windows: [1,2]→5, [2,3]→8).

One-line explanation: Slide a window of length k and compute its weighted score.

```
static int maxSubarrayScore(int[] a, int k) {
    int maxScore = 0;

    for (int i = 0; i <= a.length - k; i++) {
        int score = 0;
        for (int j = 0; j < k; j++)
            score += (j + 1) * a[i + j];

        maxScore = Math.max(maxScore, score);
    }
    return maxScore;
}
```

10) Index Sort

Description: Count the elements that are not already equal to their 1-based index, using distinct misplaced values.

Input: arr=[1,3,3,4] **Output:** Output: 3 (illustrative).

One-line explanation: Store distinct values that are in the wrong position, then subtract their count from n.

```
static int indexSort(int[] a) {
    HashSet<Integer> unique = new HashSet<>();

    for (int i = 0; i < a.length; i++) {
        if (a[i] != i + 1)
            unique.add(a[i]);
    }
    return a.length - unique.size();
}
```

11) String Swap

Description: Apply operation 1: swap first/last characters; operation 2: rotate the string by half.

Input: s="abcd", ops=[1,2] **Output:** Output: "dcba".

One-line explanation: Perform the requested string operation one by one.

```
static String stringSwap(String s, int[] ops) {
    String ans = s;

    for (int op : ops) {
        if (op == 1) {
            if (ans.length() > 1)
                ans = ans.substring(ans.length()-1)
                    + ans.substring(1, ans.length()-1)
                    + ans.substring(0,1);
        } else if (op == 2) {
            int n = ans.length() / 2;
            ans = ans.substring(n) + ans.substring(0, n);
        }
    }
    return ans;
}
```

12) Minimum Distance of ASCII Character

Description: For each target character, find the character in the source string with the smallest ASCII distance and add that distance.

Input: s="abc", targets="az" **Output:** Output: 2.

One-line explanation: Compare ASCII differences and keep the closest source character for each target.

```
static int minDistance(String s, String targets) {
    int total = 0;

    for (char target : targets.toCharArray()) {
        char best = s.charAt(0);
        int min = Math.abs(target - best);

        for (int i = 1; i < s.length(); i++) {
            int d = Math.abs(target - s.charAt(i));
            if (d < min) {
                min = d;
                best = s.charAt(i);
            }
        }
        total += Math.abs(target - best);
    }
    return total;
}
```

13) Olivia's Garden

Description: For every value n in the array, sum (a[i]-n) over the array and return the maximum such sum.

Input: arr=[1,2,4] **Output:** Output: 5.

One-line explanation: Treat each array value as n and calculate the total difference from all elements.

```
static int oliviasGarden(int[] a) {
    int max = 0;

    for (int n : a) {
        int sum = 0;
        for (int x : a)
            sum += x - n;
        max = Math.max(max, sum);
    }
    return max;
}
```

14) Robo Race

Description: Two racers start at positions c1 and c2 and repeatedly add their step sizes until their positions meet.

Input: c1=2,c2=3,d1=2,d2=1 **Output:** Output: 4.

One-line explanation: Advance the racer currently behind until both positions become equal.

```
static int robotLap(int c1, int c2, int d1, int d2) {
    int a = c1, b = c2;

    while (true) {
        if (a == b) return a;
        if (a < b) a += d1;
        else b += d2;
    }
}
```

15) Arduino

Description: Build cumulative sums of prefixes of the input array and return the maximum cumulative value.

Input: arr=[1,2,3] **Output:** Output: 6.

One-line explanation: The code repeatedly adds each prefix sum and returns the largest running total.

```
static int arduino(int n, int[] a) {
    int cnt = 0, max = Integer.MIN_VALUE;

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++)
            cnt += a[j];
        max = Math.max(max, cnt);
    }
    return max;
}
```

16) Solve Equation

Description: Count positive integer triples i,j,k up to sqrt(N) satisfying $i^2+j^2+k^2+ij+jk+ki = N$.

Input: N=6 **Output:** Output: 1 for (1,1,1).

One-line explanation: Try all positive triples within sqrt(N) and test the equation.

```
static int solveEquation(int n) {
    int cnt = 0;
    int limit = (int)Math.sqrt(n);

    for (int i = 1; i <= limit; i++)
        for (int j = 1; j <= limit; j++)
            for (int k = 1; k <= limit; k++)
                if (i*i + j*j + k*k + i*j + j*k + k*i == n)
                    cnt++;

    return cnt;
}
```

17) Flip Division Game

Description: Flip one bit at a time in N and return the first bit position whose resulting number is divisible by the given divisor.

Input: N=8, divisor=4 **Output:** Output: 1.

One-line explanation: Use XOR with a bit mask to flip one bit and test divisibility.

```
static int flipDivGame(int n, int divisor) {
    for (int ans = 1; ans <= 32; ans++) {
        int flip = n ^ (1 << (ans - 1));
        if (flip % divisor == 0)
            return ans;
    }
    return -1;
}
```

18) Jump and Jump

Description: Repeat input2 times: if the current value is positive subtract input3, otherwise add input3; return the absolute result.

Input: input1=5,input2=3,input3=2 **Output:** Output: 1.

One-line explanation: Each jump changes the value toward zero by input3.

```
static int jumpAndJump(int input1, int input2, int input3) {
    int ans = input1;

    for (int i = 0; i < input2; i++) {
        if (ans > 0) ans -= input3;
        else ans += input3;
    }
    return Math.abs(ans);
}
```

19) Marbles

Description: Starting from N, repeatedly subtract A or B when possible and count the distinct reachable values.

Input: N=5,A=2,B=3 **Output:** Output: 6 (0 through 5 are reachable).

One-line explanation: Use a set so each reachable marble count is stored only once.

```
static int marbles(int n, int a, int b) {
    HashSet<Integer> set = new HashSet<>();
    set.add(n);

    for (int step = 0; step < n; step++) {
        ArrayList<Integer> cur = new ArrayList<>(set);

        for (int x : cur) {
            if (x - a >= 0) set.add(x - a);
            if (x - b >= 0) set.add(x - b);
        }
    }
    return set.size();
}
```

20) Diwali Contest

Description: Start with 90 points and spend 5*i minutes for each solved question while the remaining time allows it.

Input: input1=3,input2=20 **Output:** Output: 92.

One-line explanation: Greedily solve questions in increasing time cost while time remains.

```
static int diwaliContest(int n, int usedTime) {
    int totalTime = 240;
    int rem = totalTime - usedTime;
    int cnt = 90;

    for (int i = 1; i <= n; i++) {
        int time = 5 * i;
        if (time <= rem) {
            cnt += 1;
            rem -= time;
        }
    }
    return cnt;
}
```

21) Reduce the Bill

Description: Flip each bit of the binary representation once and return the minimum resulting decimal value.

Input: input1=10 (1010) **Output:** Output: 2 (0010).

One-line explanation: Flip each binary bit exactly once, convert back to decimal, and keep the minimum.

```
static int reduceTheBill(int n) {
    String bits = Integer.toBinaryString(n);
    int min = Integer.MAX_VALUE;

    for (int i = 0; i < bits.length(); i++) {
        char[] b = bits.toCharArray();
        b[i] = (b[i] == '0') ? '1' : '0';
        int value = Integer.parseInt(new String(b), 2);
        min = Math.min(min, value);
    }
    return min;
}
```

22) Magic String

Description: Return the minimum number of changes needed to make all characters equal by keeping the most frequent character.

Input: s="aabcc" **Output:** Output: 2.

One-line explanation: Keep the most frequent character and change every other character.

```
static int magicString(String s) {
    HashMap<Character, Integer> freq = new HashMap<>();

    for (char ch : s.toCharArray())
        freq.put(ch, freq.getOrDefault(ch, 0) + 1);

    int max = 0;
    for (int v : freq.values())
        max = Math.max(max, v);

    return s.length() - max;
}
```

23) Class Room

Description: Assign each student their preferred seat if free; otherwise find the nearest free seat by scanning right, then left.

Input: preferences=[2,2,1] **Output:** Output: [2,3,1].

One-line explanation: Try the preferred seat first, then scan to the right and finally to the left.

```
static int[] classRoom(int seatsCount, int[] pref) {
    int[] seats = new int[seatsCount];
    int[] res = new int[pref.length];

    for (int i = 0; i < pref.length; i++) {
        int p = pref[i] - 1;

        if (seats[p] == 0) {
            seats[p] = 1;
            res[i] = p + 1;
            continue;
        }

        boolean assigned = false;
        for (int j = p; j < seatsCount; j++) {
            if (seats[j] == 0) {
                seats[j] = 1;
                res[i] = j + 1;
                assigned = true;
                break;
            }
        }

        if (!assigned) {
            for (int j = 0; j < p; j++) {
                if (seats[j] == 0) {
                    seats[j] = 1;
                    res[i] = j + 1;
                    break;
                }
            }
        }
    }
    return res;
}
```

24) Sum of Array

Description: Build a list of prefix GCD values, sort it, pair the smallest half with the reversed largest half, sum the pairwise GCDs.

Input: arr=[2,4,6,8] **Output:** Output: 4.

One-line explanation: Compute prefix GCDs, sort them, then pair opposite ends and add their GCD.

```
static int sumOfArray(int[] a) {
    ArrayList<Integer> res = new ArrayList<>();

    for (int i = 0; i < a.length; i++) {
        int g = 0;
        for (int j = 0; j <= i; j++)
            g = gcd(g, a[j]);
        res.add(g);
    }

    Collections.sort(res);
    int ans = 0, n = res.size();

    for (int i = 0; i < n / 2; i++)
        ans += gcd(res.get(i), res.get(n - 1 - i));

    return ans;
}

static int gcd(int a, int b) {
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return Math.abs(a);
}
```

25) Birthday Party

Description: Return the GCD of two inputs; the source labels this as the maximum number of equal groups/people.

Input: input1=12,input2=18 **Output:** Output: 6.

One-line explanation: The greatest common divisor gives the largest equal grouping factor.

```
static int birthdayParty(int a, int b) {
    return gcd(a, b);
}

static int gcd(int a, int b) {
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

26) Prefix-Suffix / Balance Array

Description: For every split position, calculate the absolute difference between prefix sum and suffix sum.

Input: arr=[1,2,3] **Output:** Output: [5,1,5].

One-line explanation: Compare the sum before each split with the sum after it.

```
static int[] balanceArray(int[] a) {
    int n = a.length;
    int[] result = new int[n];

    for (int i = 1; i <= n; i++) {
        int prefix = 0, suffix = 0;

        for (int j = 0; j < i; j++) prefix += a[j];
        for (int j = i; j < n; j++) suffix += a[j];

        result[i - 1] = Math.abs(prefix - suffix);
    }
    return result;
}
```

27) Reduce Till Zero

Description: Repeatedly subtract the smaller number from the larger until the second value becomes zero; return the first value.

Input: a=18,b=12 **Output:** Output: 6.

One-line explanation: This is the subtraction form of the Euclidean GCD algorithm.

```
static int reduceTillZero(int a, int b) {
    while (b > 0) {
        if (a < b) {
            int t = a; a = b; b = t;
        } else {
            b = a - b;
            a = a - b;
        }
    }
    return a;
}
```

28) Boring Arrays

Description: Sort the array and repeatedly subtract the smallest remaining value from the largest remaining value.

Input: arr=[1,2,4,7] **Output:** Output: 8.

One-line explanation: Sort first, then pair the largest with the smallest and add the differences.

```
static int boringArray(int[] a) {
    Arrays.sort(a);
    int score = 0, i = 0, j = a.length - 1;

    while (i < j) {
        score += a[j] - a[i];
        i++;
        j--;
    }
    return score;
}
```

29) Tilling Cost

Description: Recursive/memoized function from the source using $f(x,y)=2x+3y+5$ and a recurrence on $(x-2,y-2)$.

Input: $x=3,y=3$ **Output:** Output: 45 (from the visible recurrence).

One-line explanation: Memoization avoids recalculating the same (x,y) state.

```
static long tillingCost(int x, int y) {
    HashMap<String, Long> memo = new HashMap<>();
    return solveTiling(x, y, memo);
}

static long func(int x, int y) {
    return 2L * x + 3L * y + 5;
}

static long solveTiling(int x, int y, HashMap<String, Long> memo) {
    if (x == 0 || y == 0) return 0;
    if (x == 1 || y == 1) return 1L * x * y * func(x, y);

    String key = x + "," + y;
    if (memo.containsKey(key)) return memo.get(key);

    long result = solveTiling(x - 2, y - 2, memo)
        + (2L * x + 2L * y - 4) * func(x, y);

    memo.put(key, result);
    return result;
}
```

30) Fellis Function

Description: Compute the recurrence $f(n)=f(n-1)+7*f(n-2)+(n/4)$, modulo 1,000,000,007, with $f(0)=f(1)=1$.

Input: $n=4$ **Output:** Output: 10.

One-line explanation: Use memoization so each recurrence value is calculated only once.

```
static long felisFunction(int n) {
    final long MOD = 1_000_000_007L;
    long[] memo = new long[n + 1];
    Arrays.fill(memo, -1);
    return felis(n, memo, MOD);
}

static long felis(int n, long[] memo, long mod) {
    if (n == 0 || n == 1) return 1;
    if (memo[n] != -1) return memo[n];

    memo[n] = (felis(n - 1, memo, mod)
        + 7 * felis(n - 2, memo, mod)
        + n / 4) % mod;
    return memo[n];
}
```

31) Grade Swap

Description: Collect characters from the requested right-side and left-side ranges, find the minimum ASCII value, and return that character.

Input: $s="abcdef"$, $start=3$, $count=2$ **Output:** Output: 'b' (illustrative).

One-line explanation: Look around the specified grade position and choose the character with minimum ASCII value.

```
static char gradeSwap(String s, int start, int count) {
    ArrayList<Character> chars = new ArrayList<>();

    for (int i = start - 1; i < start - 1 + count && i < s.length(); i++)
        chars.add(s.charAt(i));

    for (int i = Math.max(0, start - 1 - count); i < start - 1; i++)
        chars.add(s.charAt(i));

    char min = chars.get(0);
    for (char ch : chars)
        if (ch < min) min = ch;

    return min;
}
```

32) Best Pairs — GCD and LCM

Description: For every pair (i,j) from 1 to max(input2), compute $\text{gcd}(i,j) \times \text{lcm}(i,j)$ and count when that product occurs in the input array.

Input: arr=[6,12] **Output:** Output: 4 (illustrative count).

One-line explanation: Enumerate pairs, calculate $\text{gcd} \times \text{lcm}$, and check whether that value exists in the input.

```
static int bestPairs(int[] a) {
    int max = 0;
    for (int x : a) max = Math.max(max, x);

    int cnt = 0;
    for (int i = 1; i <= max; i++) {
        for (int j = 1; j <= max; j++) {
            long g = gcd(i, j);
            long l = (long)i / g * j;
            long product = g * l;

            for (int x : a)
                if (x == product) {
                    cnt++;
                    break;
                }
        }
    }
    return cnt;
}

static int gcd(int a, int b) {
    while (b != 0) {
        int t = a % b; a = b; b = t;
    }
    return a;
}
```

33) Fellis Function (duplicate in source)

Description: Same recurrence as the earlier Fellis Function entry.

Input: n=4 **Output:** Output: 10.

One-line explanation: The iterative DP version avoids recursion while keeping the same recurrence.

```
static long felisFunction2(int n) {
    final long MOD = 1_000_000_007L;
    long[] dp = new long[Math.max(2, n + 1)];
    dp[0] = dp[1] = 1;

    for (int i = 2; i <= n; i++)
        dp[i] = (dp[i-1] + 7 * dp[i-2] + i / 4) % MOD;

    return dp[n];
}
```

34) Surprise Pair Debate

Description: Sort the array and inspect mirrored pairs whose sum is divisible by K; return the smallest relevant value.

Input: A=[1,3,5,7], K=4 **Output:** Output: 1 (illustrative).

One-line explanation: Sort, pair opposite ends, and test each pair's sum modulo K.

```
static int surpriseDebate(int[] a, int k) {
    Arrays.sort(a);
    int n = a.length;
    int result = -1;

    for (int i = 0; i < n / 2; i++) {
        if ((a[i] + a[n - 1 - i]) % k == 0) {
            int v = Math.min(a[i], a[n - 1 - i]);
            result = (result == -1) ? v : Math.min(result, v);
        }
    }

    if (n % 2 == 1 && (2 * a[n / 2]) % k == 0) {
        int v = a[n / 2];
        result = (result == -1) ? v : Math.min(result, v);
    }

    return result;
}
```

35) Pyramid Sum

Description: Build a value using the source recurrence: start with input1 and add 2*i*n while increasing n.

Input: input1=3 **Output:** Output: 17.

One-line explanation: The loop walks down the pyramid levels and adds the corresponding level contribution.

```
static int pyramidSum(int input1) {
    int res = input1;
    int n = 2;

    for (int i = input1 - 1; i >= 1; i--) {
        res += 2 * i * n;
        n++;
    }

    return res;
}
```

36) Reduce Till Zero (duplicate)

Description: Same subtraction-based reduction as entry 27.

Input: a=18,b=12 **Output:** Output: 6.

One-line explanation: Repeated subtraction reduces the pair to their GCD.

```
static int reduceTillZero2(int a, int b) {
    while (b > 0) {
        if (a < b) {
            int t = a; a = b; b = t;
        } else {
            int t = a - b;
            a = b;
            b = t;
        }
    }

    return a;
}
```

37) Toss and Score

Description: Add 2 for each H and subtract 1 for each non-H; after three consecutive H values, return immediately.

Input: toss="HHTHTH" **Output:** Output: 3.

One-line explanation: Track the score and the current consecutive-H count.

```
static int tossAndScore(String toss) {
    int score = 0, hc = 0;

    for (char ch : toss.toCharArray()) {
        if (ch == 'H') {
            hc++;
            score += 2;
            if (hc == 3) return score;
        } else {
            score -= 1;
            hc = 0;
        }
    }
    return score;
}
```

38) Qualifying Score

Description: Compute differences input4[i]-input3[i], sort descending, sum the top input2 values, then qualify if the score is at least 35.

Input: input2=2, diffs=[20,15,10] **Output:** Output: Qualified 35.

One-line explanation: Take the largest top scores from the difference array and compare their sum with 35.

```
static String qualifyingScores(int n, int top, int[] a, int[] b) {
    int[] diff = new int[n];

    for (int i = 0; i < n; i++)
        diff[i] = b[i] - a[i];

    Arrays.sort(diff);

    int score = 0;
    for (int i = n - 1; i >= Math.max(0, n - top); i--)
        score += diff[i];

    String status = score >= 35 ? "Qualified" : "Disqualified";
    return status + " " + score;
}
```